Nav2 / BehaviorTree 原始对话 Transcript

说明：

1. 这是根据当前会话逐轮整理出的原始对话式 transcript。
2. 采用 User / Assistant 轮次格式，保留了原问题和回答主内容。
3. 与系统内部不可见提示无关，只包含本轮你我之间的主要对话内容。

User:
我已经安装了clang-format，我如何确定现在vscode是通过clang-format进行格式化而非自带的C/C++呢？

Assistant:
可以直接这样确认：

1. 看配置，`C_Cpp.formatting` 是否为 `clangFormat`。
2. 看 VS Code 的 `Output -> C/C++` 日志，是否出现 `Formatting with clang-format` 或具体 clang-format 路径。
3. 做一个风格特征测试，比如设置 `BreakBeforeBraces: Allman` 或固定 `IndentWidth`，然后观察格式化结果。
4. 如需确认使用的是你安装的那个可执行文件，可通过 `which clang-format` 找到真实路径并设置 `C_Cpp.clang_format_path`。

User:
我觉得我的clang-format有点问题，为什么第9行这么长还不换行？

Assistant:
因为样式里设置了 `ColumnLimit: 0`。它的含义是关闭按列宽换行，所以即使代码很长，clang-format 也不会因为超列而主动拆行。若想自动换行，改成 `80`、`100` 或 `120` 这类常见值。

User:
即使我这里写了.clang-tidy，但是我的工作空间目录下只有.clang-format文件，一样能其作用？

Assistant:
不能。`.clang-format` 只控制格式化，`.clang-tidy` 只控制静态检查规则。两者互不替代。

User:
解释这一句语法，为什么构造SaySomething需要name和config两个参数，但是这里之提供了一个name

Assistant:
这里不是直接构造 `SaySomething` 对象，而是在注册一种节点类型：
`factory.registerNodeType<SaySomething>("SaySomething"
);`
真正构造对象发生在工厂读取 XML 并创建树实例时，框架内部才会调用 `SaySomething(name,
 config)`。

User:
要用这个需要引入什么头文件？

Assistant:
`BT::TreeObserver` 需要：
`#include <behaviortree_cpp/loggers/bt_observer.h>`

User:
运行 logger 报错：
`[main_tree_to_execute] was not specified correctly`
为什么会有这个报错？

Assistant:
因为 `logger.xml` 里定义了多棵 `<BehaviorTree>`，但 `<root>` 没写
`main_tree_to_execute="..."`。当用
`createTreeFromFile(...)` 且 XML 中有多棵树时，必须显式指定主树。

User:
`<AlwaysFailure name="failing_action"/>` 这里的 name
是一个输入 port 的名字吗？

Assistant:
不是，`name` 是节点实例名，不是 port 名。它主要用于日志、调试、可视化和路径标识。

User:
之前这里说过 `("ApproachObject")` 要和 xml 中的 `<CheckBattery
name="check_battery"/>` 一致，那么 name 所对应的
`check_battery` 可以随便取？

Assistant:
对，`name="check_battery"` 这种实例名通常可以随便取。真正必须和注册字符串一致的是
XML 标签名，比如 `<CheckBattery/>` 或 `<ApproachObject/>`。

User:
`<SaySomething message="before last_action"/>` 相对于带
name 的写法，会有什么区别吗？

Assistant:
有区别，但通常不是功能性区别，而是实例命名区别。没写 `name`
时通常会使用默认实例名，更不利于日志和调试区分；写了 `name` 更方便识别。

User:
这个匿名函数中，`BT::TreeNode& self` 是函数参数？

Assistant:
是。它是 lambda 的形参，表示当前执行到的这个行为树节点对象引用。

User:
我不太懂 `mock_test.cpp` 在实现什么东西，他的替换逻辑是怎么样的？

Assistant:
它在演示 BehaviorTree.CPP 的 substitution / mock 测试机制：不改原始
XML，而是在创建树之前给工厂加替换规则，把某些节点替换成测试节点或带脚本的测试配置。

User:
`registerBehaviorTreeFromText(xml_main);` 和
`createTreeFromFile(...)` 有区别吗？

Assistant:
有。前者是把 XML 中定义的树注册进工厂；后者是从文件读 XML 并立即创建一棵可执行树实例。

User:
`ReactiveSequence` 和普通 `Sequence` 有什么区别？

Assistant:
`ReactiveSequence` 每次 tick 都会从第一个子节点重新检查前置条件；普通
`Sequence` 更倾向于从当前 RUNNING 的子节点继续执行，不会每次都回头检查前面已成功的节点。

User:
`<Precondition if="battery_ok"
else="RUNNING"><MoveToGoal/></Precondition>` 这里 else
是什么意思？如果 `battery_ok` 为 false，下面 `<MoveToGoal>` 就不执行了？

Assistant:
对。条件为 false 时，`MoveToGoal` 不执行，整个 `Precondition`
节点直接返回你配置的 `RUNNING`。

User:
如果第一次 tick 的时候条件满足，因此 `MoveToGoal` 开始执行，然后会继续
tick，这时候如果条件不满足，就会返回 RUNNING，我的理解对吗？

Assistant:
对。后续 tick 会再次检查条件；如果条件变为 false，就不再 tick
`<MoveToGoal/>`，而是 `Precondition` 自己直接返回 `RUNNING`。

User:
这里这个 `MoveToGoal` 在第一次被 tick 后会一直执行吗？还是需要每 tick
一次再执行一次？假设 `MoveToGoal` 会占用一会儿时间的情况下。

Assistant:
后者。长时动作节点不是“第一次 tick 一下就自己后台一直跑完”，而是依赖后续行为树不断 tick
它，未完成时持续返回 `RUNNING`。

User:
如果条件都满足，`MoveToGoal` 需要执行一段时间，这段时间行为树的 tick 会继续执行然后得到
`RUNNING` 吗？`MoveToGoal` 会阻塞 tick 吗？

Assistant:
如果它是正确实现的长时动作节点，那么每次 tick 都会快速返回
`RUNNING`，不会阻塞整个行为树；如果在 `tick()` 或 `onRunning()`
里写了阻塞逻辑，比如直接 `sleep` 很久，就会阻塞该次 tick。

User:
结合这里来看，每次 `onRunning()` 都会延时，这是否就拖慢了整个 tick 的节奏？

Assistant:
对。比如 `sleep_for(10ms)` 发生在 `onRunning()` 内部时，这 10ms
就属于当前 tick 的执行时间，会直接拖慢整个 tick 节奏。

User:
假设这是一个长程执行的功能，例如 nav2 的某个路径规划器，他们的实现中也是需要阻塞 tick 的吗？

Assistant:
不是。长时任务通常应设计成：首次 tick 发起请求，后续每次 tick 快速检查是否完成，未完成就返回 `RUNNING`，而不是在一次 tick 里阻塞等待到底。

User:
Fallback 和 Sequence 的区别是什么？

Assistant:
`Sequence` 是 AND 语义：一路成功才算成功，遇到失败就停。`Fallback` 是 OR / backup 语义：一路尝试，遇到成功就停，只有全失败才失败。

User:
我这样写对吗？`return` 是节点的输出端口

Assistant:
要看节点 `providedPorts()` 是否真的声明了名为 `return` 的输出端口。更常见和更合理的命名是 `error_code`、`result` 等。是否正确取决于节点定义，而不是 XML 随便写什么都行。

User:
什么时候用 `:=`，什么时候用 `=`，什么时候不用 `{}`？

Assistant:
经验法则：
- `{var}`：用于 XML 端口绑定到 blackboard。
- `:=`：第一次创建变量并赋值。
- `=`：给已存在变量重新赋值。
- 在脚本表达式如 `_skipIf`、`_post`、`<Script code="...">` 里通常不写 `{}`，直接写变量名。

User:
看官方文档 ROS2 integration 那个例子，本质上是写一个使用 BT 的 ActionClient 来取代原来的 client 吗？

Assistant:
更准确地说，不是取代 ROS2 action client，而是把 `rclcpp_action` client 封装成一个 BT 节点，让它能被行为树调度。

User:
`#include <behaviortree_ros2/bt_action_node.hpp>` 和

`#include "nav2_behavior_tree/bt_action_node.hpp"`
有什么区别？

Assistant:
前者是更通用的 ROS2 + BehaviorTree 集成层；后者是 Nav2 自己的一层封装，依赖 Nav2 的 blackboard、server timeout、loop duration 等约定，更偏 Nav2 专用。

User:
为什么 nav2 中的 plugin 实现不是直接用 `nav2_behavior_tree/bt_action_node.hpp`，而有的地方是 `BT::ActionNodeBase`？

Assistant:
因为不是所有叫 action 的 BT 节点都对应 ROS action client。如果只是本地同步计算、blackboard 读写、简单处理，继承 `BT::ActionNodeBase` 就够了。只有真正要包装 ROS2 action client 的节点，才适合继承 `nav2_behavior_tree::BtActionNode<ActionT>`。

User:
为什么官方例子里有 `onResultReceived`、`onFailure`，但我看 `navigate_to_pose_action.hpp` 中并没有？

Assistant:
因为它们不是同一个基类接口。官方文档的例子继承的是 `behaviortree_ros2::RosActionNode<ActionT>`，而 Nav2 用的是自己实现的 `nav2_behavior_tree::BtActionNode<ActionT>`，提供的是 `on_tick()`、`on_wait_for_result()`、`on_success()`、`on_aborted()`、`on_cancelled()` 等不同的 hook。

User:
在 `compute_path_to_pose_action.cpp` 里，我的理解是 `"goal"` 是 BT 的 port，`goal_` 是对应 action 的 goal 对象，对吗？

Assistant:
对，更准确地说：
- `"goal"` 是 BT 输入端口名
- `goal_` 是 `nav2_msgs::action::ComputePathToPose::Goal` 类型的对象

- `goal_.goal` 是这个 ROS action goal 消息里的字段

User:
整个 action goal 是不是就是这个自定义 action？

Assistant:
不是。一个 action 类型分为三部分：`Goal`、`Result`、`Feedback`。`nav2_msgs::action::ComputePathToPose` 是整个 action 类型；`nav2_msgs::action::ComputePathToPose::Goal` 只是其中的 goal 部分。

User:
`FollowPath` 的 `on_tick()` 只是获取参数，那它是怎么把 path 发给 controller server 的？

Assistant:
`on_tick()` 只负责把端口值填入 `goal_`。真正发送在基类 `BtActionNode<ActionT>::tick()` 里，它在合适时机调用 `send_new_goal()`，最终通过 `action_client_->async_send_goal(goal_, ...)` 发给 action server `"follow_path"`。

User:
为什么 controller server 会读取 `follow_path` action 里的 id？

Assistant:
因为 `controller_server` 自己就是 `FollowPath` action 的 server。它不是“去读取别的 server 的 id”，而是在处理 client 发来的 `FollowPath` goal，而 `controller_id` 本来就是这个 goal 里的字段。

User:
FollowPath 怎么知道这个 Goal 要发送到哪个 controller_server 上？通常不是需要一个字符串表示某个 server 名吗？

Assistant:
是的，这个名字就是 `"follow_path"`。在 `FollowPathAction` 的 builder 里被硬编码为 ROS action 名；而 `controller_server` 创建的 action server 名也正是 `"follow_path"`，因此两边对上。

User:

ROS2 允许 ActionServer 和 ActionClient 同名吗？还是说同名是必须的？

Assistant:
更准确地说，不是“对象同名”，而是它们使用的 action
名必须一致才能通信。节点名可以不同，类名也可以不同；但 action 名比如 `"follow_path"`
必须一致。

User:
这个 action 是什么东西？是不是 server/client 创建时根据字符串创建出来的？

Assistant:
要分三层：
- `.action` 文件定义的是接口类型，例如
`nav2_msgs::action::FollowPath`
- server/client 是使用这个类型写出的代码对象
- `"follow_path"` 是这个 action 在 ROS 图里的实例名
因此，不是运行时“创建出一个新的 action 类型”，而是把某个既有 action 类型绑定到某个名字上。

User:
为什么 `NavigateToPose` 是插件形式产生的？它是被谁调用的？

Assistant:
`NavigateToPoseNavigator` 是 `bt_navigator` 通过
pluginlib 加载的 navigator 插件。`bt_navigator` 持有 `pluginli
b::ClassLoader<nav2_core::NavigatorBase>`，加载实现了
`NavigatorBase` 接口的插件，如 `NavigateToPoseNavigator`、`Nav
igateThroughPosesNavigator`。

User:
`NavigateToPose` 再上层还有东西吗？`bt_navigator`？再上层还有吗？

Assistant:
有。`bt_navigator` 是宿主节点；再上层还有外部客户端，比如 RViz、你的任务系统、调度器等。

User:
`bt_navigator` 和 `BehaviorTreeNavigator` 是什么关系？

Assistant:
`bt_navigator`
是宿主生命周期节点；`BehaviorTreeNavigator<ActionT>` 是 navigator

插件继承的通用基类。实际链条是：
`BtNavigator` 加载 `NavigatorBase` 插件
-> 具体插件如 `NavigateToPoseNavigator`
-> 它继承自 `BehaviorTreeNavigator<nav2_msgs::action::Navigate
ToPose>`

User:
从 `bt_navigator` 开始，把 `on_activate()` 这一条通路告诉我。

Assistant:
生命周期管理器调用 `BtNavigator::on_activate()`，后者遍历所有
navigator 插件，调用
`navigators_[i]->on_activate()`。运行时多态进入
`BehaviorTreeNavigator<ActionT>::on_activate()`，其中先激活
`bt_action_server_`，再调用子类的 `activate()`。

User:
是不是可以这样说：`bt_navigator` 加载
`NavigateToPoseNavigator`，后者有 `BtActionServer`，其内部最终用
`rclcpp_action::create_server<ActionT>()` 创建
`"navigate_to_pose"` 这个 server？

Assistant:
对。更准确地说，`NavigateToPoseNavigator` 继承自 `BehaviorTreeNav
igator<nav2_msgs::action::NavigateToPose>`，基类在配置阶段创建
`BtActionServer<ActionT>`；后者内部包装
`SimpleActionServer<ActionT>`，最终调用
`rclcpp_action::create_server<ActionT>(...,
"navigate_to_pose", ...)`。

User:
然后这里我卡住了。发出 action request 之后，是发给
`NavigateToPoseNavigator` 所持有的那个 `NavigateToPose` 类型的
server 吗？这个 request 并不是外部发出的而是 subscriber 触发回调后自己发出的？

Assistant:
要分两条入口：
- 外部 action client 直接发给 `navigate_to_pose`
- `/goal_pose` topic 触发 `NavigateToPoseNavigator` 内部的
`self_client_`，把 topic 转成一次对同一个 `navigate_to_pose`

server 的 action request
两条入口最后都会汇入同一个顶层 `NavigateToPose` action server。

User:
为什么会有 `bt_action_server_impl.hpp`，不是
`bt_action_server.hpp` 吗？

Assistant:
因为 `BtActionServer` 是模板类。模板实现通常必须放在头文件中，所以常见写法是：
- `bt_action_server.hpp` 放声明
- `bt_action_server_impl.hpp` 放模板成员函数实现

User:
`onGoalReceived`、`onLoop` 等这些函数，是如何传递给
`BtActionServer` 的？它们是不是最终 `SimpleActionServer` 里使用的
`handle_goal`、`handle_cancel`、`handle_accepted`？

Assistant:
不是一回事。
- `handle_goal` / `handle_cancel` / `handle_accepted`
是 ROS2 action server 的底层协议回调，由 `SimpleActionServer` 实现
- `onGoalReceived` / `onLoop` / `onPreempt` /
`onCompletion` 是 Nav2 在 `BtActionServer`
这层自己定义的业务回调，用来把“收到 goal 后做什么、每轮 BT loop 做什么、preempt
时做什么、结束时做什么”挂接进去

User:
有没有什么系统性的架构文档？

Assistant:
建议阅读：
- Nav2 官方 behavior tree walkthrough
- `nav2_bt_navigator/README.md`
- `nav2_behavior_tree/README.md`
- 工作区里的 `Pipeline.md`

User:
好的，帮我补全 `Pipeline.md`

Assistant:

已补全，新增了 `bt_navigator`、navigator
plugin、`BtActionServer`、行为树 XML、`FollowPath` 链等系统化说明。

User:
在 Nav2 官方文档中提到了 `Behavior Plugin`、`Behavior Tree
Plugin`、`Navigator Plugin` 等等，我想知道他们具体对应哪一个模块？例如
`Navigator Plugin` 的一个实现就是 `NavigateToPose`，其他的呢？

Assistant:
对应关系如下：
- Navigator Plugin：`nav2_bt_navigator`，例子 `NavigateToP
oseNavigator`、`NavigateThroughPosesNavigator`
- Behavior Tree
Plugin：`nav2_behavior_tree/plugins/...`，例子 `FollowPath
Action`、`ComputePathToPoseAction`、`SpinAction`
- Behavior Plugin：`nav2_behaviors/plugins/...`，例子
`Spin`、`BackUp`、`Wait`、`DriveOnHeading`

User:
在之前的学习中，从来没碰见过 `Behavior Plugin`，它在哪里被用到了？

Assistant:
它不由 `bt_navigator` 直接加载，而是由 `behavior_server`
加载。行为树中的某些 BT 节点，如 `<Spin/>`、`<BackUp/>`、`<Wait/>`，对应的
BT 插件会通过 action client 把请求发给 `behavior_server`，再由其内部的
Behavior Plugin 执行。

User:
为什么还需要这个 `behavior_server` 呢？所有的动作都交给
`controller_server` 不行吗？

Assistant:
不行。`controller_server` 的职责是沿路径连续控制；`behavior_server`
负责恢复动作或独立行为，如原地旋转、倒车、等待、沿当前朝向走一段。这些不属于路径跟踪问题。

User:
如果说 `bt_navigator` 加载 `NavigateToPoseNavigator` 然后创建了
`navigate_to_pose` 这个 server，这个 `navigate_to_pose`
server 和 `behavior_server` 是同一级的吗？如果是的话，这个
`behavior_server` 是谁创建的？

Assistant:
是同一级，都是 Nav2 bringup 启动的独立 ROS2 节点。`behavior_server`
不是由 `bt_navigator` 创建，而是由 launch 文件直接启动。

User:
带我走一下从 launch 文件开始，启动 `behavior_server` 并且加载这些
`spin`、`wait` 等插件的过程，以及如何和行为树被驱动到这些入口时如何跳转到插件的执行的过程。

Assistant:
链路是：
- launch 文件启动 `behavior_server`
- 参数文件里声明 `behavior_plugins: ["spin", "backup",
"drive_on_heading", "assisted_teleop", "wait"]`
- `behavior_server` 在配置阶段通过 pluginlib 加载对应的
`nav2_behaviors::Spin`、`Wait` 等 Behavior Plugin
- 行为树运行到 XML `<Spin/>` 时，对应的 BT plugin 是 `SpinAction`
- `SpinAction` 把参数组装成 ROS action goal，发给 `"spin"` 这个
action server
- `behavior_server` 接收 goal，并把执行交给内部已加载的
`nav2_behaviors::Spin`

User:
之前我们讨论过，`follow_path_action.cpp/hpp`
本身只是实现了一个可供行为树调用的入口，将行为树的参数接收之后再通过 action client
发布出去，真正的执行在 `controller_server` 那边。那现在对于
`spin.cpp`，它并非是一个对应 xml 的入口，而是一个插件？这里我理解不了。

Assistant:
对。要区分两层：
- `spin_action.cpp/hpp` 是 Behavior Tree Plugin，是 XML
`<Spin/>` 的入口
- `nav2_behaviors/plugins/spin.cpp` 是 Behavior
Plugin，是 `behavior_server` 内部真正执行旋转动作的插件

User:
我明白了。`nav2_behaviors::Spin` 也是包了一层
`ActionServer`，底层仍然是标准 ROS2 action server 在起作用。不同的是
`NavigateToPose` 在 accepted 后进入的是运行行为树，而 `Spin` 在
accepted 后进入的是 `TimedBehavior::execute()`，其中执行 `onRun`

和 `onCycleUpdate()`。我的理解对吗？

Assistant:
对。两条链结构相似：
- `NavigateToPose`：accepted 后运行的是行为树
- `Spin`：accepted 后运行的是具体行为插件逻辑

User:
`spin.cpp` 中持有的 action server 在哪里被命名为 `"spin"`？另外它的
`goal_accepted` 实现在哪里？

Assistant:
`"spin"` 这个名字来自 `behavior_server` 把插件 ID `spin` 作为
name 传给 `TimedBehavior::configure()`，后者用这个名字创建
`SimpleActionServer`。
`goal_accepted` 不是 `spin.cpp` 自己实现的，而是
`nav2_util::SimpleActionServer` 实现的。`TimedBehavior`
只是在创建 server 时把执行回调绑定成了 `TimedBehavior::execute()`。

User:
现在小结一下。也就是说在 Nav2 中，navigator 会创建一个 action server
来处理例如 `NavigateToPose` 这样的 request。收到 request
后开始运行行为树，行为树中运行 `ComputePath` 或 `FollowPath`
这样的插件，这些插件从行为树获取参数，然后通过内部的 action client 发出 request 给
`follow_path` 这样的 action server（即
`controller_server`），后者再调用内部的控制算法插件等发布 `cmd_vel`
命令。我的理解对吗？

Assistant:
对。再补一句最重要的分层：
- `NavigateToPose` 是顶层任务 action
- `ComputePathToPose` / `FollowPath` / `Spin`
是行为树内部调用的下游 action
- controller plugin / behavior plugin 是各自 server
内部的算法插件，不是 action server

User:
帮我把刚才问的几个比较重要的问题，还有小结的部分，添加进 `Pipeline.md` 中。

Assistant:

已补充进 `Pipeline.md`，并增加了 `NavigateToPose` 顶层 action ser
ver、`BtActionServer`、`SimpleActionServer`、`behavior_se
rver`、`FollowPath` 链、`Spin` 链等说明。

User:
帮我做成并列图并且加进 `.md` 中。

Assistant:
已补充两条并列链：
- `FollowPath`: behavior tree -> BT plugin ->
`controller_server` -> controller plugin
- `Spin`: behavior tree -> BT plugin ->
`behavior_server` -> behavior plugin

User:
把以上所有对话记录输出为一个 pdf

Assistant:
首次导出的 PDF 因内置字体不支持中文而乱码。随后改为使用本机可用的中文字体重新导出，并按 User / Assistant transcript 格式生成。